Apuração XML — código completo comentado

app.py · 1437 linhas · gerado em 11/11/2026 01:52

Aplicativo Streamlit que lê XMLs de NF-e/NFC-e, consolida por CFOP, mês e cliente, exporta Excel/PDF
e guarda cada relatório gerado em um banco SQLite, consultável na página "Relatórios salvos".
Comentários e docstrings aparecem em verde itálico. Emojis do código são exibidos como □ (limitação da fonte).

SEÇÕES DO ARQUIVO

Configuração da página e estilo visual	linha 241 · pág. 4
Leitura dos XMLs	linha 278 · pág. 5
Consolidações	linha 538 · pág. 8
Exportação	linha 598 · pág. 8
Banco de dados (histórico de relatórios) — SQLite ou SQL Server	linha 791 · pág. 10
Interface	linha 1265 · pág. 16

FUNÇÕES

def cfop_description()	Descrição oficial do CFOP ('' quando não catalogado).	linha 231 · pág. 4
def local_name()	Retorna o nome da tag XML sem o namespace.	linha 280 · pág. 5
def first_text()	Localiza a primeira ocorrência de uma tag pelo nome local.	linha 285 · pág. 5
def to_float()	Converte texto numérico do XML em float (aceita vírgula; erro	linha 296 · pág. 5
def child_value()	Valor numérico de um filho direto (pelo nome local).	linha 304 · pág. 5
def group_sum()	Soma <tag> dentro do grupo (ICMS, PIS, COFINS) do imposto do i	linha 312 · pág. 5
def parse_items()	Extrai os itens (det) com CFOP, valor e tributos.	linha 324 · pág. 5
def parse_xml()	Extrai identificação, destinatário, totais e itens (CFOP) da N	linha 360 · pág. 6
def collect_entries()	Expande os envios (XML e ZIP) em uma lista de (nome, função_de	linha 462 · pág. 7
def process_uploads()	Processa o lote (até MAX_FILES) com barra de progresso.	linha 498 · pág. 7
def aggregate_reports()	Consolida as notas por mês+cliente, por mês e por cliente (som	linha 540 · pág. 8
def aggregate_cfop()	Soma os itens por CFOP (geral e por mês) e acrescenta a descri	linha 560 · pág. 8
def build_excel()	Gera o Excel (bytes) com uma aba para cada consolidado, notas	linha 600 · pág. 8
def fmt_doc()	Formata CNPJ (00.000.000/0000-00) ou CPF (000.000.000-00).	linha 647 · pág. 9
def emitentes_lista()	Linhas 'Razão social — CNPJ: ...' dos emitentes com vendas a c	linha 657 · pág. 9
def _emitentes_pdf()	Texto (com marcação do ReportLab) dos emitentes, para o cabeça	linha 675 · pág. 9
def brl()	Formata número como moeda brasileira: 1234.5 -> R$ 1.234,50.	linha 685 · pág. 9
def _table_style()	Estilo padrão das tabelas do PDF (cabeçalho azul, linhas zebra	linha 690 · pág. 9
def build_pdf()	Gera o relatório PDF (bytes, A4 paisagem): indicadores, soma p	linha 705 · pág. 10
def _config()	Lê uma configuração: variável de ambiente primeiro, depois st.	linha 799 · pág. 11
def _descricao_banco()	Texto do banco em uso para exibir na tela (nunca mostra usuári	linha 825 · pág. 11
def _tipo_sql()	Tipo da coluna: números -> REAL/FLOAT; textos -> TEXT/NVARCHAR	linha 876 · pág. 11
def _comandos_ddl()	Lista de comandos CREATE TABLE / CREATE INDEX (só cria o que a	linha 883 · pág. 11
def db_conn()	Abre a conexão (SQLite ou SQL Server), garante as tabelas e fa	linha 957 · pág. 12
def _exec()	Executa um comando (parâmetros com '?', igual nos dois bancos)	linha 991 · pág. 13
def _df_sql()	Executa uma consulta e devolve um DataFrame (funciona com sqli	linha 1001 · pág. 13
def _linhas()	Converte o DataFrame em tuplas prontas para INSERT (NaN -> NUL	linha 1008 · pág. 13
def save_report()	Grava o relatório (resumo, notas e itens) no banco.	linha 1021 · pág. 13
def list_reports()	Lista os relatórios salvos (resumo), do mais recente para o ma	linha 1073 · pág. 14
def load_report()	Retorna (meta, df, df_itens) com as mesmas colunas geradas no	linha 1083 · pág. 14
def rename_report()	Altera a descrição (nome) de um relatório salvo.	linha 1100 · pág. 14
def delete_report()	Exclui o relatório e, em cascata, suas notas e itens.	linha 1106 · pág. 14
def testar_conexao()	Abre o banco, cria as tabelas se faltarem e faz uma consulta s	linha 1112 · pág. 14
def _carregar_consolidado()	Carrega um relatório do banco e recalcula os consolidados (res	linha 1124 · pág. 14
def render_historico()	Página 'Relatórios salvos': consulta, reabertura, download e e	linha 1134 · pág. 14

```python
"""Apuração XML — aplicativo Streamlit para NF-e / NFC-e.

Fluxo geral:
  1. O usuário envia XMLs (ou ZIPs) de notas fiscais.
  2. Cada XML é lido (parse_xml) e vira uma nota + seus itens (com CFOP).
  3. Os dados são consolidados por CFOP, mês e cliente (aggregate_*).
  4. Os relatórios são exibidos na tela e exportados em Excel e PDF.
  5. Cada relatório gerado é gravado automaticamente em um banco de dados
     (SQLite ou SQL Server) e pode ser consultado depois na página "Relatórios salvos".

Banco de dados (variáveis de ambiente ou .streamlit/secrets.toml):
  APURACAO_DB_URL  string de conexão ODBC do SQL Server (exige pyodbc + ODBC Driver 18)
  APURACAO_DB      caminho do arquivo SQLite (.db), p.ex. para abrir no DB Browser
  Sem nenhuma das duas, usa o SQLite "apuracao.db" ao lado do app.py.

Executar com:  streamlit run app.py
"""
import hashlib
import io
import os
import re
import sqlite3
import zipfile
import xml.etree.ElementTree as ET
from contextlib import contextmanager
from datetime import datetime
from pathlib import Path
from decimal import Decimal, InvalidOperation

import pandas as pd
import streamlit as st
from reportlab.lib import colors
from reportlab.lib.pagesizes import landscape, A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from xml.sax.saxutils import escape
from reportlab.lib.units import mm
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle

# Limite de XMLs processados por lote (os excedentes são ignorados com aviso).
MAX_FILES = 8000
# Colunas numéricas somadas nos consolidados por nota (mês, cliente) ...
METRICS = ["Faturamento (R$)", "ICMS (R$)", "PIS (R$)", "COFINS (R$)"]
# ... e nos consolidados por item (CFOP).
CFOP_METRICS = ["Valor Itens (R$)", "ICMS (R$)", "PIS (R$)", "COFINS (R$)"]

# Descrições dos CFOPs mais comuns (tabela oficial). As entradas 2xxx/3xxx usam o
# mesmo texto das 1xxx e as saídas 6xxx/7xxx o mesmo texto das 5xxx.
# CFOPs fora da tabela usam a "natureza da operação" (natOp) informada no XML.
# Para acrescentar um CFOP, basta incluir uma linha "codigo|descrição".
# Tabela de CFOPs no formato "codigo|descrição", uma por linha.
_CFOP_TABLE = """
1101|Compra para industrialização ou produção rural
1102|Compra para comercialização
1111|Compra para industrialização de mercadoria recebida anteriormente em consignação industrial
1113|Compra para comercialização, de mercadoria recebida anteriormente em consignação mercantil
1116|Compra para industrialização ou produção rural originada de encomenda para recebimento futuro
1117|Compra para comercialização originada de encomenda para recebimento futuro
1120|Compra para industrialização, em venda à ordem, já recebida do vendedor remetente
1121|Compra para comercialização, em venda à ordem, já recebida do vendedor remetente
1122|Compra para industrialização em que a mercadoria foi remetida pelo fornecedor ao industrializador sem transitar pelo estabelecimento adquirente
1124|Industrialização efetuada por outra empresa
1125|Industrialização efetuada por outra empresa quando a mercadoria remetida para utilização no processo de industrialização não transitou pelo estabelecimento adquirente da mercadoria
1151|Transferência para industrialização ou produção rural
1152|Transferência para comercialização
1153|Transferência de energia elétrica para distribuição
1154|Transferência para utilização na prestação de serviço
1201|Devolução de venda de produção do estabelecimento
1202|Devolução de venda de mercadoria adquirida ou recebida de terceiros
1203|Devolução de venda de produção do estabelecimento, destinada à Zona Franca de Manaus ou Áreas de Livre Comércio
1204|Devolução de venda de mercadoria adquirida ou recebida de terceiros, destinada à Zona Franca de Manaus ou Áreas de Livre Comércio
1205|Anulação de valor relativo à prestação de serviço de comunicação
1206|Anulação de valor relativo à prestação de serviço de transporte
1207|Anulação de valor relativo à venda de energia elétrica
1208|Devolução de produção do estabelecimento, remetida em transferência
1209|Devolução de mercadoria adquirida ou recebida de terceiros, remetida em transferência
1251|Compra de energia elétrica para distribuição ou comercialização
1401|Compra para industrialização ou produção rural em operação com mercadoria sujeita ao regime de substituição tributária
1403|Compra para comercialização em operação com mercadoria sujeita ao regime de substituição tributária
1406|Compra de bem para o ativo imobilizado cuja mercadoria está sujeita ao regime de substituição tributária
1407|Compra de mercadoria para uso ou consumo cuja mercadoria está sujeita ao regime de substituição tributária
1408|Transferência para industrialização ou produção rural em operação com mercadoria sujeita ao regime de substituição tributária
1409|Transferência para comercialização em operação com mercadoria sujeita ao regime de substituição tributária
1410|Devolução de venda de produção do estabelecimento em operação com produto sujeito ao regime de substituição tributária
1411|Devolução de venda de mercadoria adquirida ou recebida de terceiros em operação com mercadoria sujeita ao regime de substituição tributária
```

```
1551|Compra de bem para o ativo imobilizado
1552|Transferência de bem do ativo imobilizado
1553|Devolução de venda de bem do ativo imobilizado
1554|Retorno de bem do ativo imobilizado remetido para uso fora do estabelecimento
1555|Entrada de bem do ativo imobilizado de terceiro, remetido para uso no estabelecimento
1556|Compra de material para uso ou consumo
1557|Transferência de material para uso ou consumo
1601|Recebimento, por transferência, de crédito de ICMS
1603|Ressarcimento de ICMS retido por substituição tributária
1901|Entrada para industrialização por encomenda
1902|Retorno de mercadoria remetida para industrialização por encomenda
1903|Entrada de mercadoria remetida para industrialização e não aplicada no referido processo
1904|Retorno de remessa para venda fora do estabelecimento
1905|Entrada de mercadoria recebida para depósito em depósito fechado ou armazém geral
1906|Retorno de mercadoria remetida para depósito fechado ou armazém geral
1907|Retorno simbólico de mercadoria remetida para depósito fechado ou armazém geral
1908|Entrada de bem por conta de contrato de comodato ou locação
1909|Retorno de bem remetido por conta de contrato de comodato ou locação
1910|Entrada de bonificação, doação ou brinde
1911|Entrada de amostra grátis
1912|Entrada de mercadoria ou bem recebido para demonstração
1913|Retorno de mercadoria ou bem remetido para demonstração
1914|Retorno de mercadoria ou bem remetido para exposição ou feira
1915|Entrada de mercadoria ou bem recebido para conserto ou reparo
1916|Retorno de mercadoria ou bem remetido para conserto ou reparo
1917|Entrada de mercadoria recebida em consignação mercantil ou industrial
1918|Devolução de mercadoria remetida em consignação mercantil ou industrial
1919|Devolução simbólica de mercadoria vendida ou utilizada em processo industrial, remetida anteriormente em consignação
 ↳ mercantil ou industrial
1920|Entrada de vasilhame ou sacaria
1921|Retorno de vasilhame ou sacaria
1922|Lançamento efetuado a título de simples faturamento decorrente de compra para recebimento futuro
1923|Entrada de mercadoria remetida por vendedor remetente, em venda à ordem
1926|Lançamento efetuado a título de reclassificação de mercadoria decorrente de formação de kit ou de sua desagregação
1932|Aquisição de serviço de transporte iniciado em unidade da Federação diversa daquela onde inscrito o prestador
1933|Aquisição de serviço tributado pelo ISSQN
1949|Outra entrada de mercadoria ou prestação de serviço não especificada
5101|Venda de produção do estabelecimento
5102|Venda de mercadoria adquirida ou recebida de terceiros
5103|Venda de produção do estabelecimento, efetuada fora do estabelecimento
5104|Venda de mercadoria adquirida ou recebida de terceiros, efetuada fora do estabelecimento
5105|Venda de produção do estabelecimento que não deva por ele transitar
5106|Venda de mercadoria adquirida ou recebida de terceiros, que não deva por ele transitar
5109|Venda de produção do estabelecimento destinada à Zona Franca de Manaus ou Áreas de Livre Comércio
5110|Venda de mercadoria adquirida ou recebida de terceiros, destinada à Zona Franca de Manaus ou Áreas de Livre Comércio
5111|Venda de produção do estabelecimento remetida anteriormente em consignação industrial
5112|Venda de mercadoria adquirida ou recebida de terceiros remetida anteriormente em consignação industrial
5113|Venda de produção do estabelecimento remetida anteriormente em consignação mercantil
5114|Venda de mercadoria adquirida ou recebida de terceiros remetida anteriormente em consignação mercantil
5115|Venda de mercadoria adquirida ou recebida de terceiros, recebida anteriormente em consignação industrial
5116|Venda de produção do estabelecimento originada de encomenda para entrega futura
5117|Venda de mercadoria adquirida ou recebida de terceiros, originada de encomenda para entrega futura
5118|Venda de produção do estabelecimento entregue ao destinatário por conta e ordem do adquirente originário, em venda à
 ↳ ordem
5119|Venda de mercadoria adquirida ou recebida de terceiros entregue ao destinatário por conta e ordem do adquirente orig
 ↳ inário, em venda à ordem
5120|Venda de mercadoria adquirida ou recebida de terceiros entregue ao destinatário pelo vendedor remetente, em venda à
 ↳ ordem
5124|Industrialização efetuada para outra empresa
5125|Industrialização efetuada para outra empresa quando a mercadoria remetida para utilização no processo de industriali
 ↳ zação não transitou pelo estabelecimento adquirente da mercadoria
5151|Transferência de produção do estabelecimento
5152|Transferência de mercadoria adquirida ou recebida de terceiros
5153|Transferência de energia elétrica
5155|Transferência de produção do estabelecimento, que não deva por ele transitar
5156|Transferência de mercadoria adquirida ou recebida de terceiros, que não deva por ele transitar
5201|Devolução de compra para industrialização ou produção rural
5202|Devolução de compra para comercialização
5203|Devolução de mercadoria recebida em consignação mercantil ou industrial
5205|Anulação de valor relativo a aquisição de serviço de comunicação
5206|Anulação de valor relativo a aquisição de serviço de transporte
5207|Anulação de valor relativo à compra de energia elétrica
5208|Devolução de mercadoria recebida em transferência para industrialização ou produção rural
5209|Devolução de mercadoria recebida em transferência para comercialização
5210|Devolução de compra para utilização na prestação de serviço
5251|Venda de energia elétrica para distribuição ou comercialização
5301|Prestação de serviço de comunicação para execução de serviço da mesma natureza
5307|Prestação de serviço de comunicação a não contribuinte
5351|Prestação de serviço de transporte para execução de serviço da mesma natureza
5352|Prestação de serviço de transporte a estabelecimento industrial
5353|Prestação de serviço de transporte a estabelecimento comercial
5357|Prestação de serviço de transporte a não contribuinte
5359|Prestação de serviço de transporte a contribuinte ou a não contribuinte quando a mercadoria transportada está dispen
 ↳ sada de emissão de nota fiscal
5401|Venda de produção do estabelecimento em operação com produto sujeito ao regime de substituição tributária, na condiç
 ↳ ão de contribuinte substituto
5402|Venda de produção do estabelecimento de produto sujeito ao regime de substituição tributária, em operação entre cont
 ↳ ribuintes substitutos do mesmo produto
5403|Venda de mercadoria adquirida ou recebida de terceiros em operação com mercadoria sujeita ao regime de substituição
 ↳ tributária, na condição de contribuinte substituto
5404|Venda de mercadoria sujeita ao regime de substituição tributária, cujo imposto já tenha sido retido anteriormente
```

```
5405|Venda de mercadoria adquirida ou recebida de terceiros em operação com mercadoria sujeita ao regime de substituição tributária, na condição de contribuinte substituído
5408|Transferência de produção do estabelecimento em operação com produto sujeito ao regime de substituição tributária
5409|Transferência de mercadoria adquirida ou recebida de terceiros em operação com mercadoria sujeita ao regime de substituição tributária
5410|Devolução de compra para industrialização ou produção rural em operação com mercadoria sujeita ao regime de substituição tributária
5411|Devolução de compra para comercialização em operação com mercadoria sujeita ao regime de substituição tributária
5412|Devolução de bem do ativo imobilizado, em operação com mercadoria sujeita ao regime de substituição tributária
5413|Devolução de mercadoria destinada ao uso ou consumo, em operação com mercadoria sujeita ao regime de substituição tributária
5414|Remessa de produção do estabelecimento para venda fora do estabelecimento em operação com produto sujeito ao regime de substituição tributária
5415|Remessa de mercadoria adquirida ou recebida de terceiros para venda fora do estabelecimento, em operação com mercadoria sujeita ao regime de substituição tributária
5451|Remessa de animal e de insumo para estabelecimento produtor
5501|Remessa de produção do estabelecimento, com fim específico de exportação
5502|Remessa de mercadoria adquirida ou recebida de terceiros, com fim específico de exportação
5503|Devolução de mercadoria recebida com fim específico de exportação
5551|Venda de bem do ativo imobilizado
5552|Transferência de bem do ativo imobilizado
5553|Devolução de compra de bem para o ativo imobilizado
5554|Remessa de bem do ativo imobilizado para uso fora do estabelecimento
5555|Devolução de bem do ativo imobilizado de terceiro, recebido para uso no estabelecimento
5556|Devolução de compra de material de uso ou consumo
5557|Transferência de material de uso ou consumo
5601|Transferência de crédito de ICMS acumulado
5603|Ressarcimento de ICMS retido por substituição tributária
5901|Remessa para industrialização por encomenda
5902|Retorno de mercadoria utilizada na industrialização por encomenda
5903|Retorno de mercadoria recebida para industrialização e não aplicada no referido processo
5904|Remessa para venda fora do estabelecimento
5905|Remessa para depósito fechado ou armazém geral
5906|Retorno de mercadoria depositada em depósito fechado ou armazém geral
5907|Retorno simbólico de mercadoria depositada em depósito fechado ou armazém geral
5908|Remessa de bem por conta de contrato de comodato ou locação
5909|Retorno de bem recebido por conta de contrato de comodato ou locação
5910|Remessa em bonificação, doação ou brinde
5911|Remessa de amostra grátis
5912|Remessa de mercadoria ou bem para demonstração
5913|Retorno de mercadoria ou bem recebido para demonstração
5914|Remessa de mercadoria ou bem para exposição ou feira
5915|Remessa de mercadoria ou bem para conserto ou reparo
5916|Retorno de mercadoria ou bem recebido para conserto ou reparo
5917|Remessa de mercadoria em consignação mercantil ou industrial
5918|Devolução de mercadoria recebida em consignação mercantil ou industrial
5919|Devolução simbólica de mercadoria vendida ou utilizada em processo industrial, recebida anteriormente em consignação mercantil ou industrial
5920|Remessa de vasilhame ou sacaria
5921|Devolução de vasilhame ou sacaria
5922|Lançamento efetuado a título de simples faturamento decorrente de venda para entrega futura
5923|Remessa de mercadoria por conta e ordem de terceiros, em venda à ordem ou em operações com armazém geral ou depósito fechado
5926|Lançamento efetuado a título de reclassificação de mercadoria decorrente de formação de kit ou de sua desagregação
5927|Lançamento efetuado a título de baixa de estoque decorrente de perda, roubo ou deterioração
5928|Lançamento efetuado a título de baixa de estoque decorrente do encerramento da atividade da empresa
5929|Lançamento efetuado em decorrência de emissão de documento fiscal relativo a operação ou prestação também registrada em equipamento Emissor de Cupom Fiscal (ECF)
5932|Prestação de serviço de transporte iniciada em unidade da Federação diversa daquela onde inscrito o prestador
5933|Prestação de serviço tributado pelo ISSQN (Nota Fiscal conjugada)
5949|Outra saída de mercadoria ou prestação de serviço não especificado
"""
# Converte a tabela de texto em dicionário {código: descrição}.
CFOP_DESC = dict(line.split("|", 1) for line in _CFOP_TABLE.strip().splitlines())
# Descrições específicas que diferem da regra 1xxx/5xxx (ex.: vendas a não contribuinte).
_CFOP_OVERRIDES = {
    "6107": "Venda de produção do estabelecimento, destinada a não contribuinte",
    "6108": "Venda de mercadoria adquirida ou recebida de terceiros, destinada a não contribuinte",
}
# Mapeia o 1º dígito do CFOP para o grupo da tabela: 1/2/3 -> entradas (1xxx); 5/6/7 -> saídas (5xxx).
_CFOP_BASE = {"1": "1", "2": "1", "3": "1", "5": "5", "6": "5", "7": "5"}


def cfop_description(cfop):
    """Descrição oficial do CFOP ('' quando não catalogado)."""
    c = str(cfop).strip()
    if c in _CFOP_OVERRIDES:
        return _CFOP_OVERRIDES[c]
    if len(c) == 4 and c.isdigit() and c[0] in _CFOP_BASE:
        return CFOP_DESC.get(_CFOP_BASE[c[0]] + c[1:], "")
    return ""

# ---------------------------------------------------------------------------
# Configuração da página e estilo visual
# ---------------------------------------------------------------------------
st.set_page_config(
    page_title="Apuração XML | Relatórios",
    page_icon="□",
    layout="wide",
)
```

```python
# CSS customizado: fundo, cartões de métricas e o banner (hero) do topo.
st.markdown("""
<style>
    .main {background: #f6f8fb;}
    .block-container {padding-top: 2rem; padding-bottom: 2rem;}
    div[data-testid="stMetric"] {
        background: white; border: 1px solid #e7ebf1;
        padding: 16px; border-radius: 12px;
    }
    .hero {
        padding: 22px 26px; border-radius: 16px;
        background: linear-gradient(110deg,#12345a,#176b87);
        color: white; margin-bottom: 18px;
    }
    .hero h1 {color:white; margin:0;}
    .hero p {color:#e7f4fa; margin:6px 0 0 0;}
</style>
""", unsafe_allow_html=True)

# Banner de título do aplicativo.
st.markdown("""
<div class="hero">
  <h1>□ Apuração XML</h1>
  <p>Importe lotes de até 8.000 arquivos NF-e/NFC-e (XML ou ZIP) e gere relatórios consolidados em Excel e PDF, inclusive por CFOP.</p>
</div>
""", unsafe_allow_html=True)


# ---------------------------------------------------------------------------
# Leitura dos XMLs
# ---------------------------------------------------------------------------
def local_name(tag):
    """Retorna o nome da tag XML sem o namespace."""
    return tag.rsplit("}", 1)[-1].split(":")[-1]


def first_text(root, tag_names, default=""):
    """Localiza a primeira ocorrência de uma tag pelo nome local."""
    wanted = set(tag_names if isinstance(tag_names, (list, tuple, set)) else [tag_names])
    for node in root.iter():
        if local_name(node.tag) in wanted and node.text:
            value = node.text.strip()
            if value:
                return value
    return default


def to_float(value):
    """Converte texto numérico do XML em float (aceita vírgula; erro -> 0.0)."""
    try:
        return float(Decimal(str(value or "0").replace(",", ".")))
    except (InvalidOperation, ValueError, TypeError):
        return 0.0


def child_value(node, tag):
    """Valor numérico de um filho direto (pelo nome local)."""
    for child in node:
        if local_name(child.tag) == tag:
            return to_float(child.text)
    return 0.0


def group_sum(imposto, group, tag):
    """Soma <tag> dentro do grupo (ICMS, PIS, COFINS) do imposto do item.
    Grupos de substituição (PISST/COFINSST) não entram, pois têm outro nome."""
    total = 0.0
    for grp in imposto:
        if local_name(grp.tag) == group:
            for node in grp.iter():
                if local_name(node.tag) == tag:
                    total += to_float(node.text)
    return total


def parse_items(root, arquivo, chave, numero, mes_ano, natop=""):
    """Extrai os itens (det) com CFOP, valor e tributos."""
    items = []
    for det in root.iter():
        if local_name(det.tag) != "det":
            continue
        prod = next((n for n in det if local_name(n.tag) == "prod"), None)
        if prod is None:
            continue
        imposto = next((n for n in det if local_name(n.tag) == "imposto"), None)
        cfop = "N/A"
        for n in prod:
            if local_name(n.tag) == "CFOP" and n.text and n.text.strip():
                cfop = n.text.strip()
                break
```

```
        valor = (
            child_value(prod, "vProd") + child_value(prod, "vFrete")
            + child_value(prod, "vSeg") + child_value(prod, "vOutro")
            - child_value(prod, "vDesc")
        )
        items.append({
            "Arquivo": arquivo,
            "Chave": chave,
            "Número": numero,
            "Mês/Ano": mes_ano,
            "CFOP": cfop,
            "Descrição CFOP": cfop_description(cfop),
            "Natureza Operação": natop,
            "Valor Itens (R$)": valor,
            "ICMS (R$)": group_sum(imposto, "ICMS", "vICMS") if imposto is not None else 0.0,
            "PIS (R$)": group_sum(imposto, "PIS", "vPIS") if imposto is not None else 0.0,
            "COFINS (R$)": group_sum(imposto, "COFINS", "vCOFINS") if imposto is not None else 0.0,
        })
    return items


def parse_xml(name, content):
    """Extrai identificação, destinatário, totais e itens (CFOP) da NF-e/NFC-e.
    Retorna (nota, itens, erro)."""
    try:
        root = ET.fromstring(content)

        ide = next((n for n in root.iter() if local_name(n.tag) == "ide"), None)
        if ide is None:
            raise ValueError("Não foi encontrado o grupo de identificação <ide>.")

        def ide_value(tag, default=""):
            for node in ide:
                if local_name(node.tag) == tag and node.text:
                    return node.text.strip()
            return default

        # Modelo do documento: 55 = NF-e, 65 = NFC-e.
        modelo_cod = ide_value("mod", "N/A")
        modelo = {"55": "NF-e", "65": "NFC-e"}.get(modelo_cod, f"Modelo {modelo_cod}")
        numero = ide_value("nNF", "N/A")
        dh_emi = ide_value("dhEmi") or ide_value("dEmi") or "N/A"
        data_emissao = dh_emi[:10] if dh_emi != "N/A" else "N/A"
        mes_ano = data_emissao[:7] if data_emissao != "N/A" else "N/A"

        # Chave de acesso (usada para evitar contagem em duplicidade)
        chave = ""
        inf = next((n for n in root.iter() if local_name(n.tag) == "infNFe"), None)
        if inf is not None:
            chave = (inf.get("Id") or "").replace("NFe", "").strip()
        if not chave:
            chave = first_text(root, "chNFe", "")

        # Destinatário (cliente): CNPJ/CPF e nome. Sem <dest> -> consumidor final.
        dest = next((n for n in root.iter() if local_name(n.tag) == "dest"), None)
        doc_cliente = "Não identificado"
        nome_cliente = "Consumidor Final"
        if dest is not None:
            for node in dest.iter():
                tag = local_name(node.tag)
                if tag in ("CNPJ", "CPF") and node.text:
                    doc_cliente = node.text.strip()
                elif tag == "xNome" and node.text:
                    nome_cliente = node.text.strip()

        # Emitente (quem vendeu). Só é guardado para vendas a consumidor final.
        emit = next((n for n in root.iter() if local_name(n.tag) == "emit"), None)
        cnpj_emit = ""
        nome_emit = ""
        if emit is not None:
            for node in emit:
                tag = local_name(node.tag)
                if tag in ("CNPJ", "CPF") and node.text and not cnpj_emit:
                    cnpj_emit = node.text.strip()
                elif tag == "xNome" and node.text:
                    nome_emit = node.text.strip()

        # Venda a consumidor final: indFinal = 1 (ou NFC-e, que é sempre consumidor final)
        consumidor_final = ide_value("indFinal") == "1" or modelo_cod == "65"
        if not consumidor_final:
            cnpj_emit = ""
            nome_emit = ""

        # Totais da nota (grupo ICMSTot): valor da NF, ICMS, PIS e COFINS.
        total = next((n for n in root.iter() if local_name(n.tag) == "ICMSTot"), None)
        values = {"vNF": 0, "vICMS": 0, "vPIS": 0, "vCOFINS": 0}
        if total is not None:
            for node in total.iter():
                tag = local_name(node.tag)
                if tag in values:
                    values[tag] = to_float(node.text)
```

```
        else:
            for tag in values:
                values[tag] = to_float(first_text(root, tag, "0"))

        # Linha da nota que alimenta as tabelas e relatórios.
        nota = {
            "Arquivo": name,
            "Chave": chave,
            "Consumidor Final": "Sim" if consumidor_final else "Não",
            "CNPJ Emitente": cnpj_emit,
            "Emitente": nome_emit,
            "Modelo": modelo,
            "Número": numero,
            "Data Emissão": data_emissao,
            "Mês/Ano": mes_ano,
            "Documento Cliente": doc_cliente,
            "Nome Cliente": nome_cliente,
            "Faturamento (R$)": values["vNF"],
            "ICMS (R$)": values["vICMS"],
            "PIS (R$)": values["vPIS"],
            "COFINS (R$)": values["vCOFINS"],
        }
        # Itens (det) da nota, cada um com seu CFOP e tributos.
        itens = parse_items(root, name, chave, numero, mes_ano, ide_value("natOp"))
        return nota, itens, None
    # Erros não derrubam o lote: o arquivo é registrado como erro e o processamento segue.
    except ET.ParseError as exc:
        return None, [], f"XML inválido ou malformado: {exc}"
    except Exception as exc:
        return None, [], str(exc)


def collect_entries(uploads, ignorar_recebidos=False):
    """Expande os envios (XML e ZIP) em uma lista de (nome, função_de_leitura).
    Com ignorar_recebidos, descarta XMLs cujo nome (ou pasta dentro do ZIP) contém 'recebidos'.
    Retorna (entradas, quantidade_ignorada)."""
    entries = []
    ignorados = 0

    # Regra do checkbox "não ler arquivos com recebidos no nome".
    def ignorar(nome):
        return ignorar_recebidos and "recebidos" in nome.lower()

    for up in uploads:
        if up.name.lower().endswith(".zip"):
            # ZIP: abre e lista os XMLs internos (leitura adiada via lambda, para economizar memória).
            try:
                zf = zipfile.ZipFile(io.BytesIO(up.getvalue()))
            except zipfile.BadZipFile:
                entries.append((up.name, None))
                continue
            for info in zf.infolist():
                low = info.filename.lower()
                if info.is_dir() or "__macosx" in low or not low.endswith(".xml"):
                    continue
                if ignorar(info.filename):
                    ignorados += 1
                    continue
                entries.append((info.filename, lambda zf=zf, info=info: zf.read(info)))
        else:
            # XML avulso.
            if ignorar(up.name):
                ignorados += 1
                continue
            entries.append((up.name, lambda up=up: up.getvalue()))
    return entries, ignorados


def process_uploads(uploads, ignorar_recebidos=False):
    """Processa o lote (até MAX_FILES) com barra de progresso.
    Retorna (df, df_itens, errors, duplicates, total_found, ignorados)."""
    entries, ignorados = collect_entries(uploads, ignorar_recebidos)
    total_found = len(entries)
    entries = entries[:MAX_FILES]  # respeita o limite por lote

    notas, itens, errors = [], [], []
    seen = set()  # chaves de acesso já lidas (evita contar a mesma nota duas vezes)
    duplicates = 0
    progress = st.progress(0.0, text="Lendo arquivos...")
    n = len(entries)
    for i, (name, loader) in enumerate(entries, start=1):
        if loader is None:
            errors.append((name, "ZIP inválido ou corrompido."))
        else:
            try:
                nota, its, error = parse_xml(name, loader())
            except Exception as exc:
                nota, its, error = None, [], str(exc)
            if nota is None:
                errors.append((name, error))
            elif nota["Chave"] and nota["Chave"] in seen:
```

```
                duplicates += 1
            else:
                if nota["Chave"]:
                    seen.add(nota["Chave"])
                notas.append(nota)
                itens.extend(its)
        # Atualiza a barra de progresso a cada 50 arquivos.
        if i % 50 == 0 or i == n:
            progress.progress(i / max(n, 1), text=f"Lendo arquivos... {i:,}/{n:,}".replace(",", "."))
    progress.empty()

    df = pd.DataFrame(notas)
    df_itens = pd.DataFrame(itens)
    return df, df_itens, errors, duplicates, total_found, ignorados


# ---------------------------------------------------------------------------
# Consolidações
# ---------------------------------------------------------------------------
def aggregate_reports(df):
    """Consolida as notas por mês+cliente, por mês e por cliente (soma de valores e contagem de notas)."""
    by_month_client = (
        df.groupby(["Mês/Ano", "Documento Cliente", "Nome Cliente"], dropna=False)
        .agg(**{m: (m, "sum") for m in METRICS}, **{"Qtd Notas": ("Número", "count")})
        .reset_index()
    )
    by_month = (
        df.groupby(["Mês/Ano"], dropna=False)
        .agg(**{m: (m, "sum") for m in METRICS}, **{"Qtd Notas": ("Número", "count")})
        .reset_index()
    )
    by_client = (
        df.groupby(["Documento Cliente", "Nome Cliente"], dropna=False)
        .agg(**{m: (m, "sum") for m in METRICS}, **{"Qtd Notas": ("Número", "count")})
        .reset_index()
    )
    return by_month_client, by_month, by_client


def aggregate_cfop(df_itens):
    """Soma os itens por CFOP (geral e por mês) e acrescenta a descrição de cada CFOP.
    Retorna (by_cfop, by_month_cfop)."""
    cols = ["CFOP", "Descrição CFOP", *CFOP_METRICS, "Qtd Itens", "Qtd Notas"]
    cols_m = ["Mês/Ano", *cols]
    if df_itens.empty:
        return pd.DataFrame(columns=cols), pd.DataFrame(columns=cols_m)
    spec = {
        **{m: (m, "sum") for m in CFOP_METRICS},
        "Qtd Itens": ("CFOP", "count"),
        "Qtd Notas": ("Arquivo", "nunique"),
    }

    # Descrição: tabela oficial; se o CFOP não estiver catalogado, usa a natureza
    # da operação mais frequente informada nos XMLs.
    def natop_mais_frequente(s):
        s = s[s.astype(str).str.strip() != ""]
        return s.mode().iat[0] if not s.empty else ""

    natop = df_itens.groupby("CFOP")["Natureza Operação"].agg(natop_mais_frequente)
    desc = {}
    for cfop in df_itens["CFOP"].unique():
        d = cfop_description(cfop)
        if not d:
            n = natop.get(cfop, "")
            d = f"{n} (natureza da operação no XML)" if n else "CFOP não catalogado"
        desc[cfop] = d

    by_cfop = df_itens.groupby("CFOP", dropna=False).agg(**spec).reset_index()
    by_cfop.insert(1, "Descrição CFOP", by_cfop["CFOP"].map(desc))
    by_cfop = by_cfop.sort_values("Valor Itens (R$)", ascending=False).reset_index(drop=True)
    by_month_cfop = df_itens.groupby(["Mês/Ano", "CFOP"], dropna=False).agg(**spec).reset_index()
    by_month_cfop.insert(2, "Descrição CFOP", by_month_cfop["CFOP"].map(desc))
    df_itens["Descrição CFOP"] = df_itens["CFOP"].map(desc)
    return by_cfop, by_month_cfop


# ---------------------------------------------------------------------------
# Exportação
# ---------------------------------------------------------------------------
def build_excel(df, by_month_client, by_month, by_client, by_cfop, by_month_cfop, df_itens):
    """Gera o Excel (bytes) com uma aba para cada consolidado, notas e itens."""
    output = io.BytesIO()
    with pd.ExcelWriter(output, engine="openpyxl") as writer:
        by_cfop.to_excel(writer, sheet_name="Soma por CFOP", index=False)
        by_month_cfop.to_excel(writer, sheet_name="CFOP por Mês", index=False)
        by_month_client.to_excel(writer, sheet_name="Mês e Cliente", index=False)
        by_month.to_excel(writer, sheet_name="Consolidado Mensal", index=False)
        by_client.to_excel(writer, sheet_name="Consolidado Cliente", index=False)
        df.to_excel(writer, sheet_name="Notas Detalhadas", index=False)
        df_itens.to_excel(writer, sheet_name="Itens por CFOP", index=False)

```

```
        # Linha de total na aba de CFOP
        ws_cfop = writer.book["Soma por CFOP"]
        if len(by_cfop):
            last = len(by_cfop) + 1
            total_row = last + 1
            ws_cfop.cell(row=total_row, column=1, value="TOTAL")
            for idx, col in enumerate(by_cfop.columns, start=1):
                if col not in ("CFOP", "Descrição CFOP"):
                    letter = ws_cfop.cell(row=1, column=idx).column_letter
                    ws_cfop.cell(row=total_row, column=idx, value=f"=SUM({letter}2:{letter}{last})")
                    ws_cfop.cell(row=total_row, column=idx).font = ws_cfop.cell(row=1, column=1).font.copy(bold=True)
            ws_cfop.cell(row=total_row, column=1).font = ws_cfop.cell(row=1, column=1).font.copy(bold=True)

        # Formatação de todas as abas: cabeçalho fixo, filtro, largura das colunas e formato R$.
        for ws in writer.book.worksheets:
            ws.freeze_panes = "A2"
            ws.auto_filter.ref = ws.dimensions
            money_cols = {c[0].column for c in ws.iter_cols(min_row=1, max_row=1)
                          if "(R$)" in str(c[0].value or "")}
            for cells in ws.columns:
                sample = cells[:2000]
                max_len = max((len(str(c.value or "")) for c in sample), default=10)
                cap = 80 if str(cells[0].value) == "Descrição CFOP" else 38
                ws.column_dimensions[cells[0].column_letter].width = min(max(max_len + 2, 12), cap)
            for row in ws.iter_rows(min_row=2):
                for cell in row:
                    if cell.column in money_cols and (
                        isinstance(cell.value, (int, float))
                        or (isinstance(cell.value, str) and cell.value.startswith("="))
                    ):
                        cell.number_format = '"R$" #,##0.00'
    output.seek(0)
    return output.getvalue()


def fmt_doc(doc):
    """Formata CNPJ (00.000.000/0000-00) ou CPF (000.000.000-00)."""
    d = str(doc or "").strip()
    if len(d) == 14 and d.isdigit():
        return f"{d[:2]}.{d[2:5]}.{d[5:8]}/{d[8:12]}-{d[12:]}"
    if len(d) == 11 and d.isdigit():
        return f"{d[:3]}.{d[3:6]}.{d[6:9]}-{d[9:]}"
    return d


def emitentes_lista(df, limite=5):
    """Linhas 'Razão social — CNPJ: ...' dos emitentes com vendas a consumidor final."""
    if df.empty or "CNPJ Emitente" not in df:
        return []
    df = df[df["CNPJ Emitente"].astype(str).str.strip() != ""]
    if df.empty:
        return []
    pares = (df[["CNPJ Emitente", "Emitente"]].drop_duplicates()
             .sort_values("CNPJ Emitente").values.tolist())
    linhas = []
    for cnpj, nome in pares[:limite]:
        doc = fmt_doc(cnpj) or "não informado"
        linhas.append(f"{nome or 'Emitente'} — CNPJ: {doc}")
    if len(pares) > limite:
        linhas.append(f"... e mais {len(pares) - limite} emitente(s)")
    return linhas


def _emitentes_pdf(df):
    """Texto (com marcação do ReportLab) dos emitentes, para o cabeçalho do PDF."""
    linhas = emitentes_lista(df)
    if not linhas:
        return "Emitente (vendas a consumidor final): nenhuma venda a consumidor final no lote"
    rotulo = "Emitente (vendas a consumidor final): " if len(linhas) == 1 else \
        "Emitentes (vendas a consumidor final):<br/>"
    return rotulo + "<br/>".join(escape(l) for l in linhas)


def brl(value):
    """Formata número como moeda brasileira: 1234.5 -> R$ 1.234,50."""
    return f"R$ {value:,.2f}".replace(",", "X").replace(".", ",").replace("X", ".")


def _table_style(font_size=None):
    """Estilo padrão das tabelas do PDF (cabeçalho azul, linhas zebradas, grade)."""
    style = [
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#164e70")),
        ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
        ("GRID", (0, 0), (-1, -1), .35, colors.HexColor("#cbd5e1")),
        ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#f1f5f9")]),
        ("PADDING", (0, 0), (-1, -1), 4),
    ]
    if font_size:
        style.append(("FONTSIZE", (0, 0), (-1, -1), font_size))
    return TableStyle(style)
```

```python


def build_pdf(df, by_month_client, by_cfop):
    """Gera o relatório PDF (bytes, A4 paisagem): indicadores, soma por CFOP e mês/cliente."""
    output = io.BytesIO()
    doc = SimpleDocTemplate(
        output, pagesize=landscape(A4),
        rightMargin=12*mm, leftMargin=12*mm, topMargin=12*mm, bottomMargin=12*mm
    )
    styles = getSampleStyleSheet()
    story = [
        Paragraph("Relatório de Apuração Fiscal — NF-e / NFC-e", styles["Title"]),
        Paragraph(f"Relatório gerado em {datetime.now().strftime('%d/%m/%Y %H:%M')}", styles["Normal"]),
        Paragraph(_emitentes_pdf(df), styles["Normal"]),
        Spacer(1, 5*mm),
    ]
    # --- Indicadores gerais ---
    totals = [
        ["Indicador", "Total"],
        ["Quantidade de notas", str(len(df))],
        ["Faturamento", brl(df["Faturamento (R$)"].sum())],
        ["ICMS", brl(df["ICMS (R$)"].sum())],
        ["PIS", brl(df["PIS (R$)"].sum())],
        ["COFINS", brl(df["COFINS (R$)"].sum())],
    ]
    t = Table(totals, colWidths=[55*mm, 45*mm])
    t.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#164e70")),
        ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
        ("GRID", (0, 0), (-1, -1), .4, colors.HexColor("#cbd5e1")),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#f1f5f9")]),
        ("PADDING", (0, 0), (-1, -1), 6),
    ]))
    story += [t, Spacer(1, 7*mm), Paragraph("Soma dos valores por CFOP", styles["Heading2"])]

    # --- Soma por CFOP ---
    desc_style = ParagraphStyle("cfop_desc", parent=styles["Normal"], fontSize=7, leading=8.5)
    cfop_data = [["CFOP", "Descrição", "Notas", "Itens", "Valor dos itens", "ICMS", "PIS", "COFINS"]]
    for _, r in by_cfop.iterrows():
        cfop_data.append([
            str(r["CFOP"]), Paragraph(escape(str(r["Descrição CFOP"])), desc_style),
            str(int(r["Qtd Notas"])), str(int(r["Qtd Itens"])),
            brl(r["Valor Itens (R$)"]), brl(r["ICMS (R$)"]),
            brl(r["PIS (R$)"]), brl(r["COFINS (R$)"]),
        ])
    if len(by_cfop):
        cfop_data.append([
            "TOTAL", "", "", str(int(by_cfop["Qtd Itens"].sum())),
            brl(by_cfop["Valor Itens (R$)"].sum()), brl(by_cfop["ICMS (R$)"].sum()),
            brl(by_cfop["PIS (R$)"].sum()), brl(by_cfop["COFINS (R$)"].sum()),
        ])
    cfop_table = Table(cfop_data, repeatRows=1,
                       colWidths=[16*mm, 92*mm, 14*mm, 14*mm, 34*mm, 30*mm, 30*mm, 30*mm])
    cfop_style = _table_style(8)
    if len(by_cfop):
        cfop_style.add("FONTNAME", (0, -1), (-1, -1), "Helvetica-Bold")
        cfop_style.add("BACKGROUND", (0, -1), (-1, -1), colors.HexColor("#e2e8f0"))
    cfop_table.setStyle(cfop_style)
    story += [
        cfop_table,
        Paragraph(
            "Valor dos itens = vProd + frete + seguro + outras despesas − desconto, somado por CFOP "
            "do item. Pode diferir do faturamento (vNF) quando há IPI, ICMS-ST ou outros acréscimos.",
            styles["Italic"],
        ),
        Spacer(1, 7*mm),
        Paragraph("Consolidado por mês e cliente", styles["Heading2"]),
    ]

    # --- Mês e cliente ---
    data = [["Mês/Ano", "Documento", "Cliente", "Notas",
             "Faturamento", "ICMS", "PIS", "COFINS"]]
    for _, row in by_month_client.iterrows():
        data.append([
            str(row["Mês/Ano"]), str(row["Documento Cliente"]),
            str(row["Nome Cliente"])[:32], str(int(row["Qtd Notas"])),
            brl(row["Faturamento (R$)"]), brl(row["ICMS (R$)"]),
            brl(row["PIS (R$)"]), brl(row["COFINS (R$)"]),
        ])
    table = Table(data, repeatRows=1, colWidths=[22*mm, 32*mm, 54*mm, 15*mm, 32*mm, 25*mm, 25*mm, 25*mm])
    table.setStyle(_table_style(7))
    story.append(table)
    doc.build(story)
    output.seek(0)
    return output.getvalue()


# ---------------------------------------------------------------------------
# Banco de dados (histórico de relatórios) — SQLite ou SQL Server
# ---------------------------------------------------------------------------
# O banco é escolhido pela configuração (variável de ambiente ou .streamlit/secrets.toml):
```

```python
#   APURACAO_DB_URL -> SQL Server. Valor = string de conexão ODBC. Exige o pacote
#                      pyodbc e o "ODBC Driver 18 for SQL Server" instalados.
#   APURACAO_DB     -> SQLite. Valor = caminho do arquivo .db (abre no DB Browser).
#   (nenhuma)       -> SQLite em "apuracao.db", ao lado do app.py.
# Se as duas estiverem definidas, vale o SQL Server.
def _config(nome):
    """Lê uma configuração: variável de ambiente primeiro, depois st.secrets."""
    valor = os.environ.get(nome)
    if valor:
        return valor
    try:
        valor = st.secrets.get(nome)
    except Exception:  # sem arquivo secrets.toml
        return None
    return valor if isinstance(valor, str) and valor else None


DB_URL = _config("APURACAO_DB_URL")
USA_SQLSERVER = bool(DB_URL)
DB_PATH = Path(_config("APURACAO_DB") or Path(__file__).with_name("apuracao.db"))

# Nomes das tabelas (no SQL Server ficam no esquema dbo).
T_REL, T_NOTAS, T_ITENS = (
    ("dbo.relatorios", "dbo.notas", "dbo.itens") if USA_SQLSERVER
    else ("relatorios", "notas", "itens")
)
# Diferenças de sintaxe para "primeira linha apenas": TOP 1 (SQL Server) x LIMIT 1 (SQLite).
_TOP1 = "TOP 1 " if USA_SQLSERVER else ""
_LIMIT1 = "" if USA_SQLSERVER else "LIMIT 1"


def _descricao_banco():
    """Texto do banco em uso para exibir na tela (nunca mostra usuário nem senha)."""
    if not USA_SQLSERVER:
        return f"SQLite · {DB_PATH}"

    def campo(*nomes):
        for nome in nomes:
            m = re.search(rf"(?i)(?:^|;)\s*{nome}\s*=\s*([^;]+)", DB_URL)
            if m:
                return m.group(1).strip()
        return "?"

    return f"SQL Server · {campo('SERVER', 'Data Source')} / {campo('DATABASE', 'Initial Catalog')}"


DB_DESCRICAO = _descricao_banco()

# (nome da coluna no DataFrame, nome da coluna no banco)
_NOTAS_COLS = [
    ("Arquivo", "arquivo"), ("Chave", "chave"), ("Consumidor Final", "consumidor_final"),
    ("CNPJ Emitente", "cnpj_emitente"), ("Emitente", "emitente"), ("Modelo", "modelo"),
    ("Número", "numero"), ("Data Emissão", "data_emissao"), ("Mês/Ano", "mes_ano"),
    ("Documento Cliente", "documento_cliente"), ("Nome Cliente", "nome_cliente"),
    ("Faturamento (R$)", "faturamento"), ("ICMS (R$)", "icms"),
    ("PIS (R$)", "pis"), ("COFINS (R$)", "cofins"),
]
_ITENS_COLS = [
    ("Arquivo", "arquivo"), ("Chave", "chave"), ("Número", "numero"), ("Mês/Ano", "mes_ano"),
    ("CFOP", "cfop"), ("Descrição CFOP", "descricao_cfop"),
    ("Natureza Operação", "natureza_operacao"), ("Valor Itens (R$)", "valor_itens"),
    ("ICMS (R$)", "icms"), ("PIS (R$)", "pis"), ("COFINS (R$)", "cofins"),
]
_NUMERICAS_DB = {"faturamento", "icms", "pis", "cofins", "valor_itens"}

# Tamanho (em caracteres) das colunas de texto no SQL Server. Textos maiores são
# cortados ao gravar, para o banco não recusar a nota por "string truncada".
_TAM_TEXTO = {
    "arquivo": 1000, "chave": 60, "consumidor_final": 10, "cnpj_emitente": 20,
    "emitente": 300, "modelo": 30, "numero": 30, "data_emissao": 10, "mes_ano": 10,
    "documento_cliente": 30, "nome_cliente": 300, "cfop": 10,
    "descricao_cfop": 600, "natureza_operacao": 300,
}

# (nome do índice, tabela, coluna)
_INDICES = [
    ("idx_notas_relatorio", "notas", "relatorio_id"),
    ("idx_itens_relatorio", "itens", "relatorio_id"),
    ("idx_relatorios_hash", "relatorios", "hash_lote"),
]


def _tipo_sql(db, sqlserver):
    """Tipo da coluna: números -> REAL/FLOAT; textos -> TEXT/NVARCHAR(n)."""
    if db in _NUMERICAS_DB:
        return "FLOAT" if sqlserver else "REAL"
    return f"NVARCHAR({_TAM_TEXTO[db]})" if sqlserver else "TEXT"


def _comandos_ddl(sqlserver=None):
    """Lista de comandos CREATE TABLE / CREATE INDEX (só cria o que ainda não existe).
```

```
    Sem argumento, usa o banco configurado; True/False força SQL Server/SQLite."""
    if sqlserver is None:
        sqlserver = USA_SQLSERVER
    cols_notas = ",\n    ".join(f"{db} {_tipo_sql(db, sqlserver)}" for _, db in _NOTAS_COLS)
    cols_itens = ",\n    ".join(f"{db} {_tipo_sql(db, sqlserver)}" for _, db in _ITENS_COLS)

    if sqlserver:
        cmds = [
            """IF OBJECT_ID(N'dbo.relatorios', N'U') IS NULL
CREATE TABLE dbo.relatorios (
    id INT IDENTITY(1,1) PRIMARY KEY,
    criado_em NVARCHAR(19) NOT NULL,
    descricao NVARCHAR(300) NOT NULL,
    hash_lote CHAR(64) NOT NULL,
    ignorar_recebidos BIT NOT NULL DEFAULT 0,
    qtd_notas INT, qtd_itens INT, qtd_clientes INT, qtd_cfops INT,
    duplicados INT, erros INT,
    faturamento FLOAT, icms FLOAT, pis FLOAT, cofins FLOAT,
    emitentes NVARCHAR(MAX)
)""",
            f"""IF OBJECT_ID(N'dbo.notas', N'U') IS NULL
CREATE TABLE dbo.notas (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
    relatorio_id INT NOT NULL REFERENCES dbo.relatorios(id) ON DELETE CASCADE,
    {cols_notas}
)""",
            f"""IF OBJECT_ID(N'dbo.itens', N'U') IS NULL
CREATE TABLE dbo.itens (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
    relatorio_id INT NOT NULL REFERENCES dbo.relatorios(id) ON DELETE CASCADE,
    {cols_itens}
)""",
        ]
        for nome, tabela, coluna in _INDICES:
            cmds.append(
                f"IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'{nome}' "
                f"AND object_id = OBJECT_ID(N'dbo.{tabela}')) "
                f"CREATE INDEX {nome} ON dbo.{tabela}({coluna})")
        return cmds

    cmds = [
        """CREATE TABLE IF NOT EXISTS relatorios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    criado_em TEXT NOT NULL,
    descricao TEXT NOT NULL,
    hash_lote TEXT NOT NULL,
    ignorar_recebidos INTEGER NOT NULL DEFAULT 0,
    qtd_notas INTEGER, qtd_itens INTEGER, qtd_clientes INTEGER, qtd_cfops INTEGER,
    duplicados INTEGER, erros INTEGER,
    faturamento REAL, icms REAL, pis REAL, cofins REAL,
    emitentes TEXT
)""",
        f"""CREATE TABLE IF NOT EXISTS notas (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    relatorio_id INTEGER NOT NULL REFERENCES relatorios(id) ON DELETE CASCADE,
    {cols_notas}
)""",
        f"""CREATE TABLE IF NOT EXISTS itens (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    relatorio_id INTEGER NOT NULL REFERENCES relatorios(id) ON DELETE CASCADE,
    {cols_itens}
)""",
    ]
    for nome, tabela, coluna in _INDICES:
        cmds.append(f"CREATE INDEX IF NOT EXISTS {nome} ON {tabela}({coluna})")
    return cmds


_esquema_pronto = False  # SQL Server: as tabelas são verificadas uma vez por execução do app


@contextmanager
def db_conn():
    """Abre a conexão (SQLite ou SQL Server), garante as tabelas e faz commit;
    em caso de erro faz rollback. A conexão é sempre fechada ao final."""
    global _esquema_pronto
    if USA_SQLSERVER:
        try:
            import pyodbc  # importado só quando o SQL Server é usado
        except ImportError as exc:
            raise RuntimeError(
                "Para usar o SQL Server instale o pacote pyodbc (pip install pyodbc) "
                "e o 'ODBC Driver 18 for SQL Server'.") from exc
        conn = pyodbc.connect(DB_URL, timeout=30)
    else:
        conn = sqlite3.connect(DB_PATH, timeout=30)
        conn.execute("PRAGMA foreign_keys = ON")  # necessário para o ON DELETE CASCADE
    try:
        if USA_SQLSERVER:
            if not _esquema_pronto:
                cur = conn.cursor()
```

```
                for cmd in _comandos_ddl():
                    cur.execute(cmd)
                conn.commit()
                _esquema_pronto = True
        else:
            conn.executescript(";\n".join(_comandos_ddl()) + ";")
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()


def _exec(conn, sql, params=()):
    """Executa um comando (parâmetros com '?', igual nos dois bancos) e devolve o cursor."""
    cur = conn.cursor()
    if params:
        cur.execute(sql, params)
    else:
        cur.execute(sql)
    return cur


def _df_sql(conn, sql, params=()):
    """Executa uma consulta e devolve um DataFrame (funciona com sqlite3 e pyodbc)."""
    cur = _exec(conn, sql, params)
    colunas = [d[0] for d in cur.description]
    return pd.DataFrame.from_records([tuple(r) for r in cur.fetchall()], columns=colunas)


def _linhas(df, cols):
    """Converte o DataFrame em tuplas prontas para INSERT (NaN -> NULL, ordem das colunas do
    banco). No SQL Server, textos acima do tamanho da coluna são cortados."""
    sub = df.reindex(columns=[c for c, _ in cols])
    sub = sub.astype(object).where(sub.notna(), None)
    if USA_SQLSERVER:
        for c, db in cols:
            tam = _TAM_TEXTO.get(db)
            if tam:
                sub[c] = sub[c].map(lambda v, t=tam: v[:t] if isinstance(v, str) else v)
    return sub.itertuples(index=False, name=None)


def save_report(df, df_itens, errors, duplicates, ignorar_recebidos, descricao=None):
    """Grava o relatório (resumo, notas e itens) no banco.
    Retorna (id, novo). Se o mesmo lote de notas já foi salvo, reaproveita o
    registro existente (novo=False) em vez de duplicar."""
    chaves = sorted(str(c or a) for c, a in zip(df["Chave"], df["Arquivo"]))
    hash_lote = hashlib.sha256("\n".join(chaves).encode("utf-8")).hexdigest()

    with db_conn() as conn:
        existente = _exec(
            conn, f"SELECT {_TOP1}id FROM {T_REL} WHERE hash_lote = ? ORDER BY id {_LIMIT1}",
            (hash_lote,)).fetchone()
        if existente:
            return existente[0], False

        # No SQL Server o id novo vem da cláusula OUTPUT; no SQLite, de cursor.lastrowid.
        saida = "OUTPUT INSERTED.id " if USA_SQLSERVER else ""
        cur = _exec(
            conn,
            f"""INSERT INTO {T_REL} (criado_em, descricao, hash_lote, ignorar_recebidos,
                   qtd_notas, qtd_itens, qtd_clientes, qtd_cfops, duplicados, erros,
                   faturamento, icms, pis, cofins, emitentes)
               {saida}VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""",
            (
                datetime.now().isoformat(timespec="seconds"),
                (descricao or f"Apuração de {datetime.now():%d/%m/%Y %H:%M}")[:300],
                hash_lote, int(bool(ignorar_recebidos)),
                len(df), len(df_itens), int(df["Documento Cliente"].nunique()),
                int(df_itens["CFOP"].nunique()) if not df_itens.empty else 0,
                int(duplicates), len(errors),
                float(df["Faturamento (R$)"].sum()), float(df["ICMS (R$)"].sum()),
                float(df["PIS (R$)"].sum()), float(df["COFINS (R$)"].sum()),
                "\n".join(emitentes_lista(df)),
            ),
        )
        rid = cur.fetchone()[0] if USA_SQLSERVER else cur.lastrowid

        for tabela, cols, dados in ((T_NOTAS, _NOTAS_COLS, df),
                                    (T_ITENS, _ITENS_COLS, df_itens)):
            if dados.empty:
                continue
            nomes = ", ".join(db for _, db in cols)
            marcas = ", ".join("?" for _ in cols)
            cur = conn.cursor()
            if USA_SQLSERVER:
                cur.fast_executemany = True  # inserção em lote muito mais rápida no pyodbc
            cur.executemany(
```

```
                f"INSERT INTO {tabela} (relatorio_id, {nomes}) VALUES (?, {marcas})",
                [(rid, *linha) for linha in _linhas(dados, cols)],
            )
    return rid, True


def list_reports():
    """Lista os relatórios salvos (resumo), do mais recente para o mais antigo."""
    with db_conn() as conn:
        return _df_sql(
            conn,
            f"""SELECT id, criado_em, descricao, qtd_notas, qtd_itens, qtd_clientes,
                      qtd_cfops, faturamento, icms, pis, cofins
                FROM {T_REL} ORDER BY id DESC""")


def load_report(rid):
    """Retorna (meta, df, df_itens) com as mesmas colunas geradas no processamento."""
    with db_conn() as conn:
        cur = _exec(conn, f"SELECT * FROM {T_REL} WHERE id = ?", (rid,))
        row = cur.fetchone()
        if row is None:
            return None, pd.DataFrame(), pd.DataFrame()
        meta = dict(zip([d[0] for d in cur.description], row))
        saida = []
        for tabela, cols in ((T_NOTAS, _NOTAS_COLS), (T_ITENS, _ITENS_COLS)):
            nomes = ", ".join(db for _, db in cols)
            d = _df_sql(
                conn, f"SELECT {nomes} FROM {tabela} WHERE relatorio_id = ? ORDER BY id", (rid,))
            saida.append(d.rename(columns={db: c for c, db in cols}))
    return meta, saida[0], saida[1]


def rename_report(rid, descricao):
    """Altera a descrição (nome) de um relatório salvo."""
    with db_conn() as conn:
        _exec(conn, f"UPDATE {T_REL} SET descricao = ? WHERE id = ?", (descricao[:300], rid))


def delete_report(rid):
    """Exclui o relatório e, em cascata, suas notas e itens."""
    with db_conn() as conn:  # notas e itens saem junto (ON DELETE CASCADE)
        _exec(conn, f"DELETE FROM {T_REL} WHERE id = ?", (rid,))


def testar_conexao():
    """Abre o banco, cria as tabelas se faltarem e faz uma consulta simples.
    Retorna (ok, mensagem)."""
    try:
        with db_conn() as conn:
            _exec(conn, f"SELECT COUNT(*) FROM {T_REL}").fetchone()
        return True, "Conexão estabelecida e tabelas verificadas."
    except Exception as exc:
        return False, f"Falha na conexão: {exc}"


@st.cache_data(show_spinner=False)
def _carregar_consolidado(rid):
    """Carrega um relatório do banco e recalcula os consolidados (resultado em cache por id)."""
    meta, df, df_itens = load_report(rid)
    if df.empty:
        return meta, df, df_itens, None, None
    agg = aggregate_reports(df)
    cfop = aggregate_cfop(df_itens)
    return meta, df, df_itens, agg, cfop


def render_historico():
    """Página 'Relatórios salvos': consulta, reabertura, download e exclusão."""
    st.subheader("□ Relatórios salvos")
    try:
        lista = list_reports()
    except Exception as exc:
        st.error(f"Não foi possível acessar o banco de dados ({DB_DESCRICAO}): {exc}")
        return
    if lista.empty:
        st.info("Nenhum relatório salvo ainda. Os relatórios gerados em "
                "\"Nova apuração\" são guardados aqui automaticamente.")
        return

    exib = pd.DataFrame({
        "Nº": lista["id"],
        "Gerado em": pd.to_datetime(lista["criado_em"]).dt.strftime("%d/%m/%Y %H:%M"),
        "Descrição": lista["descricao"],
        "Notas": lista["qtd_notas"],
        "Clientes": lista["qtd_clientes"],
        "CFOPs": lista["qtd_cfops"],
        "Faturamento": lista["faturamento"].map(brl),
        "ICMS": lista["icms"].map(brl),
        "PIS": lista["pis"].map(brl),
        "COFINS": lista["cofins"].map(brl),
```

```
    })
    st.dataframe(exib, use_container_width=True, hide_index=True)

    ids = lista["id"].tolist()
    ultimo = st.session_state.get("ultimo_relatorio")
    rotulos = dict(zip(exib["Nº"], exib["Gerado em"] + " — " + exib["Descrição"]))
    sel = st.selectbox(
        "Abrir relatório", ids, index=ids.index(ultimo) if ultimo in ids else 0,
        format_func=lambda i: f"#{i} · {rotulos[i]}",
    )

    meta, df, df_itens, agg, cfop = _carregar_consolidado(sel)
    if meta is None or df.empty:
        st.warning("Este relatório não possui notas gravadas.")
        return
    by_month_client, by_month, by_client = agg
    by_cfop, by_month_cfop = cfop

    # --- Renomear / excluir ---  (confirmação obrigatória para excluir)
    col_nome, col_del = st.columns([3, 1])
    with col_nome:
        novo_nome = st.text_input("Descrição do relatório", value=meta["descricao"],
                                  key=f"nome_{sel}")
        if novo_nome.strip() and novo_nome.strip() != meta["descricao"]:
            if st.button("□ Salvar descrição"):
                rename_report(sel, novo_nome.strip())
                _carregar_consolidado.clear()
                st.rerun()
    with col_del:
        st.write("")
        confirmar = st.checkbox("Confirmar exclusão", key=f"conf_{sel}")
        if st.button("□ Excluir relatório", disabled=not confirmar):
            delete_report(sel)
            _carregar_consolidado.clear()
            if st.session_state.get("ultimo_relatorio") == sel:
                st.session_state.pop("ultimo_relatorio")
            st.rerun()

    st.caption(
        f"Gerado em {datetime.fromisoformat(meta['criado_em']):%d/%m/%Y %H:%M}"
        + (f" · {meta['duplicados']} duplicado(s) ignorado(s)" if meta["duplicados"] else "")
        + (f" · {meta['erros']} arquivo(s) com erro" if meta["erros"] else "")
    )
    for linha in (meta["emitentes"] or "").splitlines():
        st.caption(f"□ {linha} (vendas a consumidor final)")

    c1, c2, c3, c4, c5 = st.columns(5)
    c1.metric("Notas processadas", f"{len(df):,}".replace(",", "."))
    c2.metric("Faturamento total", brl(df["Faturamento (R$)"].sum()))
    c3.metric("ICMS total", brl(df["ICMS (R$)"].sum()))
    c4.metric("Clientes", f"{df['Documento Cliente'].nunique():,}".replace(",", "."))
    c5.metric("CFOPs distintos", f"{by_cfop['CFOP'].nunique():,}".replace(",", "."))

    tab0, tab1, tab2, tab3, tab4 = st.tabs([
        "Soma por CFOP", "Por mês e cliente", "Consolidado mensal",
        "Por cliente", "Notas detalhadas",
    ])
    with tab0:
        st.dataframe(by_cfop, use_container_width=True, hide_index=True)
        if not by_cfop.empty:
            st.bar_chart(by_cfop.set_index("CFOP")[["Valor Itens (R$)"]])
            with st.expander("CFOP por mês"):
                st.dataframe(by_month_cfop, use_container_width=True, hide_index=True)
    with tab1:
        st.dataframe(by_month_client, use_container_width=True, hide_index=True)
    with tab2:
        st.dataframe(by_month, use_container_width=True, hide_index=True)
        st.bar_chart(by_month.set_index("Mês/Ano")[["Faturamento (R$)"]])
    with tab3:
        st.dataframe(by_client, use_container_width=True, hide_index=True)
    with tab4:
        busca = st.text_input("Filtrar notas (chave, número, cliente ou documento)",
                              key=f"busca_{sel}").strip().lower()
        notas_view = df
        if busca:
            mask = pd.Series(False, index=df.index)
            for col in ("Chave", "Número", "Documento Cliente", "Nome Cliente"):
                mask |= df[col].astype(str).str.lower().str.contains(busca, regex=False)
            notas_view = df[mask]
            st.caption(f"{len(notas_view):,} nota(s) encontrada(s).".replace(",", "."))
        st.dataframe(notas_view, use_container_width=True, hide_index=True)

    # --- Downloads (gerados sob demanda a partir dos dados gravados) ---
    st.subheader("Exportar relatórios")
    chave_arq = f"arquivos_{sel}"
    if st.button("⚙ Gerar Excel e PDF deste relatório"):
        with st.spinner("Gerando Excel e PDF..."):
            st.session_state[chave_arq] = (
                build_excel(df, by_month_client, by_month, by_client,
                            by_cfop, by_month_cfop, df_itens),
                build_pdf(df, by_month_client, by_cfop),
```

```
            )
    if chave_arq in st.session_state:
        xlsx_h, pdf_h = st.session_state[chave_arq]
        d1, d2 = st.columns(2)
        d1.download_button(
            "⬇ Baixar relatório Excel", data=xlsx_h,
            file_name=f"relatorio_apuracao_{sel}.xlsx",
            mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
            use_container_width=True, key=f"dl_x_{sel}")
        d2.download_button(
            "⬇ Baixar relatório PDF", data=pdf_h,
            file_name=f"relatorio_apuracao_{sel}.pdf",
            mime="application/pdf", use_container_width=True, key=f"dl_p_{sel}")


# ---------------------------------------------------------------------------
# Interface
# ---------------------------------------------------------------------------
# Navegação entre as duas páginas do app (menu lateral).
pagina = st.sidebar.radio("Navegação", ["Nova apuração", "Relatórios salvos"], key="pagina")
# Informa qual banco está em uso e permite testar a conexão (útil ao configurar o SQL Server).
with st.sidebar.expander("□ Banco de dados"):
    st.caption(DB_DESCRICAO)
    if st.button("Testar conexão"):
        _ok, _msg = testar_conexao()
        (st.success if _ok else st.error)(_msg)
if pagina == "Relatórios salvos":
    render_historico()
    st.stop()

ignorar_recebidos = st.checkbox(
    "Não ler arquivos XML com \"recebidos\" no nome",
    value=False,
    help="Ignora XMLs cujo nome contenha \"recebidos\" (em qualquer combinação de maiúsculas/minúsculas). "
         "Em arquivos ZIP, vale também para o nome da pasta dentro do ZIP.",
)

# Área de envio dos arquivos XML/ZIP.
uploads = st.file_uploader(
    f"Selecione arquivos XML de NF-e/NFC-e ou um/mais ZIPs (até {MAX_FILES:,} XMLs por lote)".replace(",", "."),
    type=["xml", "zip"], accept_multiple_files=True,
    help="Para lotes grandes, compacte os XMLs em um arquivo .zip e envie-o: é bem mais rápido."
)

if uploads:
    # A "assinatura" (nomes + tamanhos + opção) identifica o lote: se não mudou, reaproveita
    # o resultado em cache (st.session_state) em vez de reprocessar a cada interação.
    signature = (tuple((u.name, u.size) for u in uploads), ignorar_recebidos)
    cached = st.session_state.get("result")
    if cached and cached["signature"] == signature:
        df, df_itens, errors, duplicates, total_found, ignorados = (
            cached["df"], cached["df_itens"], cached["errors"],
            cached["duplicates"], cached["total_found"], cached["ignorados"],
        )
        by_month_client, by_month, by_client = cached["agg"]
        by_cfop, by_month_cfop = cached["cfop"]
        xlsx, pdf = cached["xlsx"], cached["pdf"]
        report_id, report_novo = cached.get("report_id"), cached.get("report_novo", False)
    else:
        # Lote novo: lê os XMLs, consolida e gera os arquivos Excel/PDF.
        df, df_itens, errors, duplicates, total_found, ignorados = process_uploads(
            uploads, ignorar_recebidos)
        if df.empty:
            by_month_client = by_month = by_client = None
            by_cfop = by_month_cfop = None
            xlsx = pdf = None
        else:
            by_month_client, by_month, by_client = aggregate_reports(df)
            by_cfop, by_month_cfop = aggregate_cfop(df_itens)
            with st.spinner("Gerando Excel e PDF..."):
                xlsx = build_excel(df, by_month_client, by_month, by_client,
                                   by_cfop, by_month_cfop, df_itens)
                pdf = build_pdf(df, by_month_client, by_cfop)
        # Grava o relatório no banco (histórico). Se já existir o mesmo lote, reaproveita o registro.
        report_id, report_novo = None, False
        if not df.empty:
            try:
                report_id, report_novo = save_report(
                    df, df_itens, errors, duplicates, ignorar_recebidos)
                st.session_state["ultimo_relatorio"] = report_id
            except Exception as exc:
                st.warning(f"Não foi possível salvar o relatório no banco de dados: {exc}")
        # Guarda tudo em cache na sessão.
        st.session_state["result"] = {
            "signature": signature, "df": df, "df_itens": df_itens,
            "errors": errors, "duplicates": duplicates, "total_found": total_found,
            "ignorados": ignorados,
            "agg": (by_month_client, by_month, by_client),
            "cfop": (by_cfop, by_month_cfop), "xlsx": xlsx, "pdf": pdf,
            "report_id": report_id, "report_novo": report_novo,
        }
```

```python

    # Avisos sobre arquivos ignorados e limite do lote.
    if ignorados:
        st.info(f"{ignorados:,} arquivo(s) com \"recebidos\" no nome não foram lidos.".replace(",", "."))

    if total_found > MAX_FILES:
        st.warning(
            f"Foram encontrados {total_found:,} XMLs, mas o limite por lote é {MAX_FILES:,}. "
            f"Somente os primeiros {MAX_FILES:,} foram processados.".replace(",", ".")
        )

    if not df.empty:
        st.success(f"{len(df):,} nota(s) processada(s) com sucesso.".replace(",", "."))
        if report_id:
            st.caption(
                f"□ Relatório #{report_id} salvo no histórico (menu lateral › Relatórios salvos)."
                if report_novo else
                f"□ Este lote já estava salvo como relatório #{report_id}; nenhum registro duplicado foi criado."
            )
        linhas_emit = emitentes_lista(df)
        for linha in linhas_emit:
            st.caption(f"□ {linha} (vendas a consumidor final)")
        if not linhas_emit:
            st.caption("Nenhuma venda a consumidor final encontrada no lote.")
        if duplicates:
            st.info(f"{duplicates:,} arquivo(s) duplicado(s) (mesma chave de acesso) foram ignorados."
                    .replace(",", "."))
        if errors:
            with st.expander(f"Ver {len(errors)} arquivo(s) não processado(s)"):
                for filename, error in errors[:500]:
                    st.warning(f"{filename}: {error}")
                if len(errors) > 500:
                    st.caption(f"... e mais {len(errors) - 500} erro(s).")

        # Indicadores principais.
        c1, c2, c3, c4, c5 = st.columns(5)
        c1.metric("Notas processadas", f"{len(df):,}".replace(",", "."))
        c2.metric("Faturamento total", brl(df["Faturamento (R$)"].sum()))
        c3.metric("ICMS total", brl(df["ICMS (R$)"].sum()))
        c4.metric("Clientes", f"{df['Documento Cliente'].nunique():,}".replace(",", "."))
        c5.metric("CFOPs distintos", f"{by_cfop['CFOP'].nunique():,}".replace(",", "."))

        # Abas com as visões consolidadas.
        st.subheader("Visão geral")
        tab0, tab1, tab2, tab3, tab4 = st.tabs([
            "Soma por CFOP", "Por mês e cliente", "Consolidado mensal",
            "Por cliente", "Notas detalhadas",
        ])
        with tab0:
            st.dataframe(by_cfop, use_container_width=True, hide_index=True)
            if not by_cfop.empty:
                st.bar_chart(by_cfop.set_index("CFOP")[["Valor Itens (R$)"]])
                with st.expander("CFOP por mês"):
                    st.dataframe(by_month_cfop, use_container_width=True, hide_index=True)
        with tab1:
            st.dataframe(by_month_client, use_container_width=True, hide_index=True)
        with tab2:
            st.dataframe(by_month, use_container_width=True, hide_index=True)
            if not by_month.empty:
                st.bar_chart(by_month.set_index("Mês/Ano")[["Faturamento (R$)"]])
        with tab3:
            st.dataframe(by_client, use_container_width=True, hide_index=True)
        with tab4:
            st.dataframe(df, use_container_width=True, hide_index=True)

        # Botões de download dos relatórios gerados.
        st.subheader("Exportar relatórios")
        d1, d2 = st.columns(2)
        with d1:
            st.download_button(
                "⬇ Baixar relatório Excel",
                data=xlsx, file_name="relatorio_apuracao_xml.xlsx",
                mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
                use_container_width=True
            )
        with d2:
            st.download_button(
                "⬇ Baixar relatório PDF",
                data=pdf, file_name="relatorio_apuracao_xml.pdf",
                mime="application/pdf", use_container_width=True
            )
    # Nenhuma nota válida: mostra os erros encontrados.
    elif errors:
        st.error("Nenhum XML válido foi processado.")
        for filename, error in errors[:200]:
            st.warning(f"{filename}: {error}")
    else:
        st.error("Nenhum XML encontrado nos arquivos enviados.")
# Tela inicial (nenhum arquivo enviado ainda).
else:
    st.info("Envie arquivos XML (ou ZIP) para visualizar os dados e habilitar os relatórios.")
```

```
    st.markdown("""
    **O aplicativo gera:**
    - Relatório Excel com abas de soma por CFOP, CFOP por mês, mês/cliente, consolidado mensal, consolidado por cliente, notas e itens detalhados.
    - Relatório PDF com indicadores gerais, soma por CFOP e consolidação por mês e cliente.
    - Indicadores de faturamento, ICMS, PIS, COFINS, quantidade de notas, clientes e CFOPs.
    - Lotes de até 8.000 XMLs (envie preferencialmente em um ZIP).
    """)
```